

CS 450/535: Assignment 01

Programming Assignments (80%)

Initial Setup

- **This assignment is in C++.**
- Copy `src/app/BasicSquare.cpp` and name it **`src/app/Assign01.cpp`**
- Change the application name from "BasicSquare" to "**Assign01**"
- Change the window title to be "Assign01: " + your SITNET ID
 - o E.g., "Assign01: realemj"
- Make a copy of the `vulkanshaders/BasicSquare` folder and name it **`vulkanshaders/Assign01`**
- Modify **`CMakeLists.txt`** by adding the following line to the end of the file:
 - o `CREATE_VULKAN_EXECUTABLE(Assign01)`
- Make sure the program configures, compiles, and runs as-is
- Add the following global:
 - o **`bool useFrontCCW = true;`**
- Add the following to the keyboard function (`key_callback`):
 - o **If `key == GLFW_KEY_Z` → toggle `useFrontCCW`**
- In `recordFrame()`, right before looping through all meshes and drawing them:
 - o If `useFrontCCW` → **`cd.buffer().setFrontFace(vk::FrontFace::eCounterClockwise);`**
 - o Otherwise → **`cd.buffer().setFrontFace(vk::FrontFace::eClockwise);`**
- In `main`, right before call to `createVulkanPipeline()`, allow front-face winding order to be dynamically set:
 - o **`pipelineCreateInfo.dynamicStates.push_back(vk::DynamicState::eFrontFace);`**
- ***Running the application and pressing "Z" should make the quad disappear and appear.***

Requirements

Both CS 450 and 535

The goal is to replace the existing `simpleShape` creation code with a mesh shape (or mesh shapes) of your own unique design.

The shape(s) created must meet the following requirements:

- The actual shape data should be stored in one or more **`pro::HostMesh<ProVertex>`** mesh objects, and these objects should be created ONCE and added to `allHostMeshes`.
- All shapes should render properly.
- All vertices MUST be STRICTLY INSIDE the viewing volume:
 - o X and Y → between -1.0 and 1.0
 - o Z → between 0.0 and 1.0

- All triangles must use COUNTER-CLOCKWISE winding order.
 - o To test this, your shape(s) should render by default, BUT disappear entirely if Z is pressed (then reappear on pressing it again, and so on...).
- At least some of the vertices should be different colors.
- Alpha channel of colors should always be 1.0f.
- Color values should be between 0.0f and 1.0f.
- Triangles may intersect, BUT they should NOT have the same Z value and be on top of each other (i.e., no Z-fighting).
- Be creative 😊 No seriously, be creative, as **your shape(s) should be unique across the class.**
 - o ***Points will be taken off proportional to the number of times I see the same or similar ideas implemented!***

CS 535 Only

You will also handle shapes with **different topologies**. To set this up:

- Add the topology as something that can be changed dynamically:
 - o `pipelineCreateInfo.dynamicStates.push_back(vk::DynamicState::ePrimitiveTopology);`
- In `renderFrame()`, given a shape that is about to be rendered with `recordDrawVulkanMesh()`, you can set the topology like this:
 - o `cd.buffer().setPrimitiveTopology(vk::PrimitiveTopology::eTriangleList);`
 - o `cd.buffer().setPrimitiveTopology(vk::PrimitiveTopology::eTriangleFan);`
 - o `cd.buffer().setPrimitiveTopology(vk::PrimitiveTopology::eTriangleStrip);`

Thus, your assignment has the following ADDITIONAL requirements:

- You must have **AT LEAST 2 shapes**:
 - o At least one shape → **triangle list**
 - o At least one shape → **triangle fan OR triangle strip**
- You do NOT have to do restarts in a fan or strip
 - o You can separate each fan or strip into separate meshes
- You must design a way to pass in the appropriate primitive topology per shape to `renderFrame()`:
 - o **Do NOT make multiple versions of `renderFrame()`!**
 - However, you obviously may change the parameters
 - o Do NOT store the topology as a global!

Screenshot (10%)

- **Take a screenshot** of the application window.
 - o *Windows:* Alt + Screenshot button OR Windows Button + Shift + S
- Copy the image into the **screenshots** folder as "Assign01.png"

README (10%)

- In README.md, change "YOUR NAME HERE" to your full name.
- Under the "Applications" section, add a subsection (same format as BasicSquare) very briefly describing what you decided to do for your shape(s).
 - o ***Also include your screenshot in the document as an (as we have done with the exercises)***

Grading

Your OVERALL assignment grade is weighted as follows:

- 80% - Programming
- 10% - Screenshot
- 10% - README

I reserve the right to take points off for not meeting the specifications in this assignment description.

The following specific penalties (applied to the WHOLE assignment) can be expected:

Problem	Maximum Penalty
Code that does not compile	60
One GIGANTIC commit for whole assignment	30
Shapes not rendered properly	30
Runs with Vulkan validation warnings/errors	15
Instance of repeated shape from another student	2.5 for every repeat
Winding order inconsistent	10
Missing/incorrect Z keyboard controls	10
Missing screenshot	10
README update missing/incorrect	10
Vertices all the same color	5
Triangles/shapes Z-fighting	5
Missing assignment Git branch	5
Failed to merge assignment branch	5
Bad/out-of-range position/color values	2.5 for each
Incorrect application name	2
Incorrect window title	2
Wrong names for branch, files, etc.	2 for each
Bad commit messages	2 for each